

ui.echart 全面详解

`ui.echart` 是 NiceGUI 框架中集成 Apache ECharts 的核心组件，用于快速创建交互式图表（含 2D/3D 图表），支持动态更新、事件监听、自定义主题等丰富功能，通过简单的 Python 语法即可调用 ECharts 强大的可视化能力。以下从核心特性、使用场景、详细配置、方法与事件等维度展开全面解析。

一、核心概述

1. 本质与定位

- 封装 Apache ECharts 库，提供 Python 式 API，无需直接编写 JavaScript 即可创建图表。
- 支持图表动态更新、事件响应、3D 渲染、主题定制等高级功能，适配从简单报表到复杂可视化的各类场景。
- 兼容 ECharts 原生配置（通过 `options` 参数），同时扩展了 NiceGUI 风格的便捷接口（如 `on_point_click`、`run_chart_method`）。

2. 核心参数（初始化时必填 / 常用）

参数名	类型	说明
<code>options</code>	dict	ECharts 核心配置字典，包含坐标轴、系列、图例等所有图表样式和数据定义（必填）。
<code>on_point_click</code>	Callable	点击图表数据点时触发的回调函数（如弹出提示、跳转等）。
<code>enable_3d</code>	bool	强制导入 <code>echarts-gl</code> 库，用于 3D 图表渲染（默认自动检测 3D 配置）。
<code>renderer</code>	str	渲染方式，可选 <code>"canvas"</code> （默认，性能优）或 <code>"svg"</code> （矢量图，高清无锯齿），v2.7.0+ 支持。
<code>theme</code>	dict / str (URL)	图表主题配置，可传入字典或返回 JSON 的 URL（缓存优化），v2.15.0+ 支持。

二、基础使用场景与示例

1. 基础静态图表（柱状图 / 折线图）

通过 `options` 定义坐标轴、数据系列，快速创建基础图表。支持柱状图（`bar`）、折线图（`line`）、饼图等 ECharts 所有原生图表类型。

示例：基础柱状图

```

from nicegui import ui

ui.echart({
    'xAxis': {'type': 'category', 'data': ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']},
    'yAxis': {'type': 'value'},
    'series': [{'type': 'bar', 'data': [20, 10, 30, 50, 40]}], # type 指定图表类型
})

ui.run()

```

2. 动态更新图表数据

修改 `echart.options` 中的数据后，图表会自动刷新，无需手动重绘。常见于实时数据监控、交互控制场景。

示例：点击按钮更新数据

```

from nicegui import ui
from random import random

# 初始化图表
echart = ui.echart({
    'xAxis': {'type': 'value'},
    'yAxis': {'type': 'category', 'data': ['A', 'B'], 'inverse': True},
    'series': [
        {'type': 'bar', 'name': 'Alpha', 'data': [0.1, 0.2]},
        {'type': 'bar', 'name': 'Beta', 'data': [0.3, 0.4]},
    ],
})

# 定义更新函数：修改第一个系列的第一个数据点为随机值
def update():
    echart.options['series'][0]['data'][0] = random()

ui.button('Update Data', on_click=update)
ui.run()

```

3. 点击数据点事件响应

通过 `on_point_click` 参数绑定回调，实现点击数据点后的交互（如弹出提示、展示详情）。

示例：点击数据点弹出提示

```
from nicegui import ui

# on_point_click 绑定 ui.notify, 点击时显示数据值
ui.echart({
    'xAxis': {'type': 'category', 'data': ['A', 'B', 'C', 'D']},
    'yAxis': {'type': 'value'},
    'series': [{'type': 'line', 'data': [20, 10, 30, 50]}],
}, on_point_click=lambda e: ui.notify(f'Clicked value: {e.args["value"]}'))

ui.run()
```

4. 动态属性配置（如格式化坐标轴标签）

在属性名前加前缀 `$`，表示该属性值为 JavaScript 表达式，支持动态格式化（如数值加单位、日期格式化）。

示例：Y 轴标签添加美元符号

```
from nicegui import ui

ui.echart({
    'xAxis': {'type': 'category', 'data': ['Jan', 'Feb', 'Mar']},
    'yAxis': {
        'type': 'value',
        'axisLabel': {'$formatter': 'value => "$" + value'} # : 表示 JS 表达式
    },
    'series': [{'type': 'line', 'data': [5, 8, 13]}],
})

ui.run()
```

5. 自定义主题

通过 `theme` 参数配置图表颜色、背景色等样式，支持传入字典（直接定义）或 URL（远程 JSON 主题，缓存优化）。可使用 [ECharts 主题编辑器](#) 生成主题。

示例：自定义主题（紫色系 + 浅背景）

```
from nicegui import ui

ui.echart({
    'xAxis': {'type': 'category', 'data': ['A', 'B', 'C']},
    'yAxis': {'type': 'value'},
    'series': [{'type': 'bar', 'data': [20, 10, 30]}],
}, theme={
    'color': ['#b687ac', '#28738a', '#a78f8f'], # 图表系列颜色
    'backgroundColor': 'rgba(254,248,239,1)', # 图表背景色
})

ui.run()
```

6. 集成 pyecharts

通过 `ui.echart.from_pyecharts()` 方法，直接将 pyecharts 图表对象转换为 NiceGUI 图表，复用 pyecharts 的配置逻辑。

示例：从 pyecharts 生成图表

```
from nicegui import ui
from pyecharts.charts import Bar
from pyecharts.options import AxisOpts
from pyecharts.commons.utils import JsCode

# 创建 pyecharts 图表
pyechart = (
    Bar()
    .add_xaxis(['A', 'B', 'C'])
    .add_yaxis('Ratio', [1, 2, 4])
    .set_global_opts(
        xaxis_opts=AxisOpts(axislabel_opts={'formatter': r'(val) => `Group ${val}`'}),
        yaxis_opts=AxisOpts(axislabel_opts={'formatter': JsCode(r'(val) => `${val}%`')}),
    )
)

# 转换为 NiceGUI 图表
ui.echart.from_pyecharts(pyechart)
ui.run()
```

7. 3D 图表渲染

当 `options` 包含 `xAxis3D`/`yAxis3D`/`zAxis3D` 时，自动启用 3D 渲染；若未检测到，可手动设置 `enable_3d=True`。支持 3D 折线图、散点图等。

示例：3D 折线图

```
from nicegui import ui

ui.echart({
    'xAxis3D': {}, # 3D X 轴
    'yAxis3D': {}, # 3D Y 轴
    'zAxis3D': {}, # 3D Z 轴
    'grid3D': {}, # 3D 网格配置
    'series': [{
        'type': 'line3D', # 3D 折线图类型
        'data': [[1, 1, 1], [2, 2, 3], [3, 3, 3], [4, 4, 5]], # 三维数据
    }],
})

ui.run()
```

8. 调用 ECharts 原生方法

通过 `run_chart_method()` 调用 ECharts 实例的原生方法（如显示加载动画、获取图表宽度、设置提示框）。方法名前加 `:` 表示参数为 JavaScript 表达式。

示例：调用原生方法控制图表

```
from nicegui import ui

echart = ui.echart({
    'xAxis': {'type': 'category', 'data': ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']},
    'yAxis': {'type': 'value'},
    'series': [{'type': 'line', 'data': [150, 230, 224, 218, 135]}],
})

# 显示加载动画
ui.button('Show Loading', on_click=lambda: echart.run_chart_method('showLoading'))
# 隐藏加载动画
ui.button('Hide Loading', on_click=lambda: echart.run_chart_method('hideLoading'))
# 获取图表宽度
async def get_width():
    width = await echart.run_chart_method('getWidth')
    ui.notify(f'Chart width: {width}px')
ui.button('Get Width', on_click=get_width)
# 设置提示框格式化 (:setOption 表示参数为 JS 表达式)
ui.button('Set Tooltip', on_click=lambda: echart.run_chart_method(
    ':setOption', r'{tooltip: {formatter: params => "$" + params.value}}'
))

ui.run()
```

9. 监听任意 ECharts 事件

通过 `on('chart:事件名')` 监听 ECharts 原生事件（如选择数据、缩放、平移），事件名需加 `chart:` 前缀。

示例：监听数据选择事件

```
from nicegui import ui

label = ui.label('Select a point with the brush tool')

# 启用刷子工具（用于选择数据），监听 selectchanged 事件
ui.echart({
    'toolbox': {'feature': {'brush': {'type': ['rect']}}}, # 刷子工具
    'brush': {}, # 启用刷子功能
    'xAxis': {'type': 'category', 'data': ['A', 'B', 'C']},
    'yAxis': {'type': 'value'},
    'series': [{'type': 'line', 'data': [1, 2, 3]}],
}).on('chart:selectchanged', lambda e: label.set_text(
    f'Selected point index: {e.args["fromActionPayload"]["dataIndexInside"]}'
))
```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
<code>options</code>	dict	可读可写，图表的核心配置（修改后自动刷新图表）。
<code>visible</code>	bool	控制图表是否可见（支持绑定，如 <code>bind_visibility</code> ）。
<code>html_id</code>	str	图表在 HTML DOM 中的唯一 ID（v2.16.0+ 支持）。
<code>style</code>	Style[Self]	图表样式（如宽度、高度），支持 Tailwind CSS（如 <code>echart.style('width: 800px')</code> ）。
<code>classes</code>	Classes[Self]	图表的 HTML 类名，用于自定义样式。

2. 核心方法

方法名	作用	参数说明
<code>from_pycharts(chart)</code>	将 pyecharts 图表对象转换为 NiceGUI 图表。	<code>chart</code> : pyecharts 实例; <code>on_point_click</code> : 可选，点击数据点回调。
<code>run_chart_method(name, *args)</code>	调用 ECharts 原生方法。	<code>name</code> : 方法名（加 <code>:</code> 表示参数为 JS 表达式）; <code>args</code> : 方法参数; <code>timeout</code> : 超时时间。
<code>on_point_click(callback)</code>	绑定数据点点击回调（等价于初始化时的 <code>on_point_click</code> 参数）。	<code>callback</code> : 接收 <code>EChartPointClickEventArguments</code> 参数的函数。
<code>on(type, handler)</code>	监听事件（支持 ECharts 原生事件，需加 <code>chart:</code> 前缀）。	<code>type</code> : 事件名（如 <code>chart:selectchanged</code> ）; <code>handler</code> : 回调函数。
<code>bind_visibility(target)</code>	绑定图表可见性到目标对象的属性（双向绑定）。	<code>target_object</code> : 目标对象; <code>target_name</code> : 属性名（默认 <code>visible</code> ）。
<code>update()</code>	手动触发图表在客户端更新（一般无需调用，修改 <code>options</code> 后自动触发）。	-
<code>delete()</code>	删除图表及所有子元素。	-

四、高级技巧与注意事项

1. 性能优化

- 主题复用：若多个图表使用同一主题，建议将主题放在远程 JSON 文件中，通过 URL 传入 `theme` 参数（浏览器缓存，减少重复加载）。
- 渲染器选择：大数据量图表用 `renderer="canvas"`（性能优先），需要高清矢量图（如印刷）用 `renderer="svg"`。
- 动态更新：避免频繁修改 `options` 顶层结构，优先修改 `series[].data` 等子属性（减少 DOM 重绘）。

2. 兼容性说明

- `renderer` 参数仅支持 v2.7.0+ 版本。
- `theme` 参数仅支持 v2.15.0+ 版本。
- `html_id` 参数仅支持 v2.16.0+ 版本。
- 3D 图表依赖 `echarts-gl` 库，若未自动导入，手动设置 `enable_3d=True`。

3. 调试技巧

- 查看 ECharts 原生配置：打印 `echart.options` 可确认当前图表的完整配置。
- 事件参数调试：在回调函数中打印 `e.args`，可查看事件的完整参数（如点击点的坐标、数据值）。
- 原生方法文档：参考 [ECharts 实例方法文档](#)，了解 `run_chart_method` 可调用的所有方法。

五、总结

`ui.echart` 是 NiceGUI 中功能最强大的可视化组件之一，兼具 ECharts 的灵活性和 Python 的简洁性。支持从简单静态图表到复杂 3D 交互图表的全场景需求，核心优势在于：

1. 零 JavaScript 基础即可使用，通过 Python 字典配置图表。
2. 动态更新、事件监听等功能开箱即用，无需手动处理 DOM。
3. 兼容 pyecharts 和 ECharts 原生生态，可复用现有配置。
4. 支持主题定制、3D 渲染等高级功能，满足专业可视化需求。

适用于数据报表、监控面板、数据分析工具等场景，是 Python 开发者快速构建交互式可视化界面的优选组件。